Module- 2

Algorithm Design techniques:

1. Brute force : A simple comprehensive search strategy that systematically explores every option until the solution is discovered.

I Selection Sort

It is a comparison based sorting algm. It sorts by repeatedly selecting the smallest / largest element from the unsorted array and swapping it with the first unsorted element.

Algorithm Selection sort (A [0...n-1])

// sort an array

// Input : an array A [0...n-1] of orderable elements

// Output : an array A[0...n-1] sorted in non decreasing order.

```
for i ← 0 to n-2 do
    min ← i
    for j ← i+1 to n-1 do
        if A[j] < A[min]
            min ← j
    swap A[i] and A[min]
```

Here, we start by scanning the entire given list to find its smallest element and

exchange it with the first element, putting the smallest element in its final position in the sorted list. Then scan the list starting with the second element to find the smallest among the last n-1 elements and exchange with the second element and so on.

Eg:-

Consider the list of element

89 (swap)	45	68	90	29	34	17	min = 89
17	45 (swap)	68	90	29	34	89	min = ~~45~~ 29
17	29	68 (swap)	90	45	34	89	min = ~~68~~ ~~45~~ 34
17	29	34	90 (swap)	45	68	89	min = ~~90~~ 45
17	29	34	45	90 (swap)	65	89	min = ~~90~~ 65
17	29	34	45	65	90 (swap)	89	min = 90
17	29	34	45	65	89	90	
17	29	34	45	65	89	90	

<u>Analysis</u>

Here, given algm is non recursive. We are using summation method to solve this

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

$$= \sum_{i=0}^{n-1} (n-\cancel{1}-i+\cancel{1}) = \sum_{i=0}^{n-1} n-i-1$$

$$= \frac{1}{2}[n(n-1)] = n^2 - n = \underline{O(n^2)}$$

Brute force string Matching

Given a string of n characters called text and a string of m characters ($m \leq n$) called the pattern, we need to find a substring of the text that matches the pattern. This algm uses the following procedure:

- align the pattern against the first m characters of the text and start matching the corresponding pairs of characters from left to right until either all the m pairs of the characters match or a mismatch pair is encountered.
- If a mismatching pair is encountered, shift the pattern one position to the right and resume the character comparison.

Eg:- T: NOBODY - NOTICED - HIM

P: NOT

```
NOT
 NOT
  NOT
   NOT
    NOT
     NOT
```

Algm BruteForce Match (T[0.. n-1], P[0.. m-1])

for $i \leftarrow 0$ to $n-m$ do

$j \leftarrow 0$

```
while j < m and P[j] = T[i+j] do
    j ← j + 1
if j = m
    return i
return -1
```

II Sequential Search

- Also known as Linear Search
- a simple algm for finding a target value within a list by checking each element one by one in the order they appear.

Algorithm Sequential Search (A[0...n], K)

// Input : An array A of n elements and a search key K.

// Output : The index of the element in A[0.. n-1] whose value is equal to K or -1 if no such element is found.

```
A[n] ← K
i ← 0
while A[i] ≠ K do
    i ← i + 1
if i < n return i
else return -1
```

Analysis

The running time of this algm can be different for the same list size n. In the worst case, when there are no matching elements or the first matching element happens to be the

last one in the list, the algm makes the largest number of key comparisons among all possible input of size n.

ie, $C_{worst}(n) = n$.

$C_{best}(n) = 1$

$C_{avg}(n) \approx n$

Exhaustive Search

- A problem solving technique that systematically examines every possible solution and select the one that satisfies the constraints.
- It works in three steps:

① generate all possible solutions

② test each soln: to check whether it satisfies the problem condition.

③ select a valid or optimal soln: among them.

- Exhaustive search is used in problems like:

① Travelling salesman problem (TSP)

② Knapsack problem

③ Assignment problem

④ Password cracking.

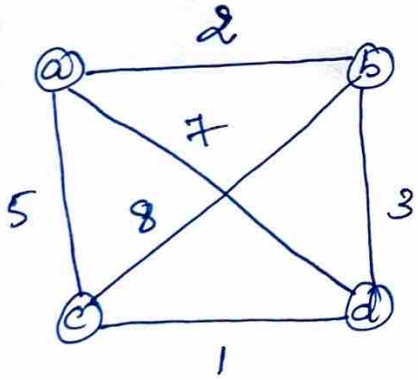
Travelling Salesman Problem:

- a classic optimization problem

Problem statement:

A salesman is given a list of n cities and the distance between each pair of cities. The objective is to find the shortest path that visits each city exactly once and returns to the starting point.

- this problem can also be stated as Hamiltonian circuit of the graph.



Tour	length
$a \to b \to c \to d \to a$	$2+8+1+7=18$
$a \to b \to d \to c \to a$	$2+3+1+5=11$
$a \to c \to d \to b \to a$	$5+1+3+2=1$
$a \to c \to b \to d \to a$	$5+8+3+7=23$

Here distance 11 is the optimal soln. and that corresponding path is the optimal path.

Knapsack Problem

Given n items of known weights $w_1, w_2, w_3, \ldots w_n$ and values $v_1, v_2, \ldots v_n$ and a knapsack of capacity W, find the most

valuable subset of the items that fits into the knapsack with maximum profit.

Eg: consider a knapsack of capacity 10, the following items are present with the corresponding weight:

$w_1 = 7$ $w_2 = 3$ $w_3 = 4$ $w_4 = 5$

$V_1 = \$42$ $V_2 = \$12$ $V_3 = \$40$ $V_4 = \$25$

Subset	Total weight	Total value
ϕ	0	\$0
{1}	$w_1 = 7$, $V_1 = \$42$	\$42
{2}	3	\$12
{3}	4	\$40
{4}	5	\$25
{1, 2}	7+3	\$42+12 = \$54
{1, 3}	7+4 = 11	not feasible
{1, 4}	7+5 = 12	not feasible
{2, 3}	7	\$52
{2, 4}	8	\$37
optimal. {3, 4}	9	\$65
{1, 2, 3}	14	not feasible
{1, 3, 4}	16	"
{2, 3, 4}	12	"